FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

BACHELOR THESIS

Jaroslav Švarc

Data-driven low-code programming system

Department of Distributed and Dependable Systems

Supervisor of the bachelor thesis: Mgr. Tomáš Petříček, Ph.D.
Study programme: Computer science

Prague 2024

I declare that I carried out this bachelor thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In ..... date ....

Author's signature

Dedication.

Title: Data-driven low-code programming system

Author: Jaroslav Švarc

Department: Department of Distributed and Dependable Systems

Supervisor: Mgr. Tomáš Petříček, Ph.D., Department of Distributed and Dependable Systems

Abstract:

Keywords: keyword, key phrase

Název práce: Data-driven low-code programming system

Autor: Jaroslav Švarc

Katedra: Katedra distribuovaných a spolehlivých systémů

Vedoucí bakalářské práce: Mgr. Tomáš Petříček, Ph.D., Katedra distribuovaných a spolehlivých systémů

Abstrakt: Abstrakt práce přeložte také do češtiny.

Klíčová slova: klíčová slova, klíčové fráze

Contents

Introduction **7**

1 Background **9**
1.1 Low-code development systems . . . 9
1.1.1 Hypercard . . . 9
1.1.2 Darklang . . . 10
1.1.3 Output-directed programming . . . 11
1.2 Web development . . . 12
1.2.1 HTML . . . 12
1.2.2 JSON . . . 12
1.2.3 Elm . . . 12
1.2.4 Fable . . . 13
1.2.5 Elmish . . . 13

2 Walktrough **14**
2.1 To-do application . . . 14

3 Design principles **15**
3.1 Data-driven approach . . . 15
3.2 Low-code editor . . . 16
3.3 Code generation . . . 16

4 Core logic **17**
4.1 Hole-based approach . . . 17
4.2 Type system and Operations . . . 17
4.2.1 JSON type . . . 17
4.2.2 RenderingCode type . . . 18
4.2.3 Data Mapping: JSON to RenderingCode . . . 19
4.3 Creation process . . . 20
4.4 Referencing input data . . . 20
4.4.1 Structual referencing . . . 20
4.4.2 Selector based approach . . . 20

5 Implementation **21**
5.1 Technologies . . . 21
5.2 Design . . . 21
5.3 User interface . . . 21
5.4 Summary . . . 21

6 Discussion **22**
6.1 7GUIs benchmarks . . . 22
6.2 Structual referencing vs Selectors . . . 22
6.3 Potential drawbacks of the approach . . . 22
6.4 Future work . . . 22
6.5 Summary . . . 22

Conclusion **23**

Bibliography **24**

List of Figures **26**

A Attachments **27**
A.1 First Attachment 27

Introduction

Since the creation of the World Wide Web, developers have created web applications mainly by writing and modifying text-based code. This approach is mainly used even today, although made easier by many different tools, languages, frameworks, and programming systems. //TODO

Problem description

Low-code programming sytems are systems that aim to significantly reduce the amount of code needed to create certain applications [1]. They can also help less experienced developers, by providing them with an easy-to-use interface, through which the developers can create program artifacts.

This thesis presents a data-driven approach for creating user interface elements of web applications. We aim to create a low-code programming system based on a data-driven approach. The approach involves first providing data to the application, which serves as the model of the application. Then, the programming system aids the developer in creating the user interface by providing suggestions based on the structure and type of the data. After creating the user interface, the developer can download the text-based representation of the application and modifying the behavior of these components and how they change the data itself. This system will allow the developers to create the user interface elements of the application without the need to write a large amount of boilerplate code.

Goals

The main goals of this thesis are the following:

1. Explore the applicability of the low-code programming system employing the data-driven approach to the creation of UI elements of web applications:
    - Define the *Data-driven* approach.
    - Define the approach in context of creating web applications based on concrete data.
    - Describe the design principles behind the approach.
    - Present an abstract description of two potential ways to implement this approach.
    - Evaluate the feasability of this approach in the context of the prototype application.
2. Create a working **prototype programming system** implementing the data-driven approach:
    - Create the prototype application that should:
        - Be browser-based.
        - Provide a low-code interface.

  - Allow the user to upload JSON data.
  - Provide context menus based on the provided data and aid the user in creating the UI elements.
  - Show a preview of the user-created parts of the web application.
  - Allow the user to download the created web application in a text-based form.
- Benchmark the prototype application on the following task:
  - Counter task from the 7GUIs Benchmark
  - Temperature converter task from the 7GUIs Benchmark
  - CRUD task from the 7GUIs Benchmark
  - Simple To-do list application

Contributions and Outline

- In Chapter 2,we showcase a creation of a user interface for a simple to-do list application.
- In Chapter 3, we describe the design principles which influenced the design of the proposed programming system.
- In Chapter 4, //TODO
- In Chapter 5, we describe the implementation specifics of the prototype application.
- In Chapter 6, //TODO

1 Background

1.1 Low-code development systems

In this section, we describe the *low-code development approach* and several systems employing it. Low-code development approach can be defined as follows:

Definition 1 (Low-code development approach)**.** *Defined by Pinho et al. [1] as "An approach for software development that uses tools that minimize (or eliminate) the number of lines of code written."*

This definition accommodates a broad spectrum of tools and development practices, such as visual programming tools, integrated development environments with code generation capabilities, development tools powered by artificial intelligence, and generators that generate source code based on template files. We focus mainly on visual programming tools employing the defined low-code approach.

Visual programming tools present a graphical user interface to the user. This graphical user interface serves as an abstraction over the development domain. It provides tools to create and modify software elements. In the following sections, we describe several existing low-code visual programming systems.

1.1.1 Hypercard

Hypercard is a low-code development system created by Bill Atkinson for the Macintosh operating system. Apple released the program in 1987 at the Macworld exposition in Boston [2]. Apple developed and maintained the program until 1998. The popularity of the program ensured that similar programs and clones of Hypercard were created after its discontinuation.

The following summary of functionality is based on a manual for the Hypercard system written by Goodman [3]. The fundamental elements that the user creates are called *cards*. Cards can hold data as text, have custom graphics, contain buttons, and implement custom behavior. Users can implement the custom behavior using a scripting language called *HyperTalk*. Then, users can group cards into *stacks*. A stack is a collection of cards with the same type of information. The program saves a stack as a single file to the disk. Finally, users can distribute and modify these stacks. Creation and modification of the card is done through the low-code interface. The program provides several options for the different card elements.

The program offers users a choice of a user level, as seen in figure 1.1. The program changes the user interface's capabilities based on the selected user level. The higher the selected user level, the more options the program enables and displays. A selected level enables all previous-level options alongside other options and functionality. Users can choose from five different user levels:

1. Browsing - enables no modification of cards and is mainly intended to be used to view the different cards inside a stack.

2. Typing - enables inserting and modifying text data inside the cards.

3. Painting - enables the creation of custom graphics inside the cards. The program provides different graphical options and tools.

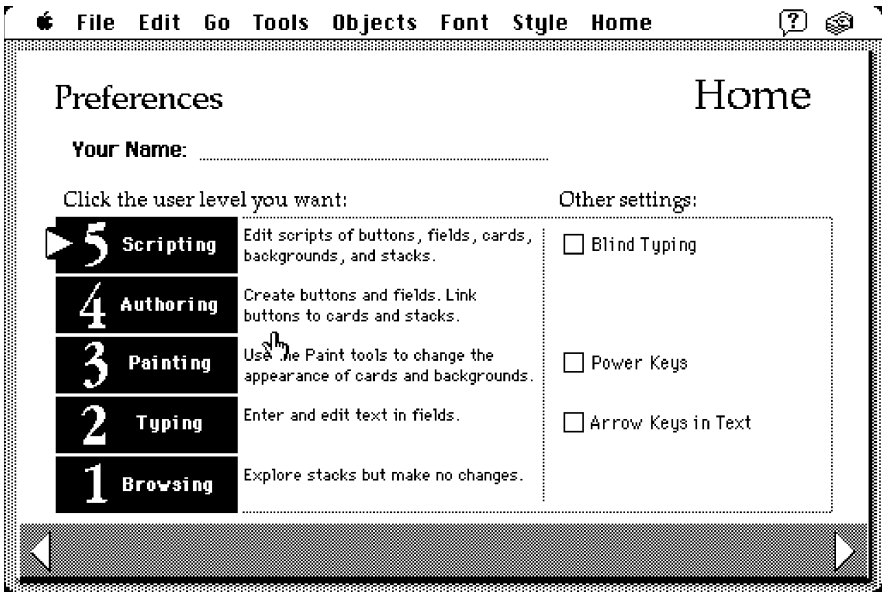

Figure 1.1 Selection of a user level in the Hypercard preferences menu.

4. Authoring - adds the ability to add buttons and fields. This way, the users can add card functionality without writing code.

5. Scripting - adds the ability to use the HyperTalk scripting language to modify the behavior of the different elements inside the cards.

This is favourable, as less experienced users can learn to use the interface more easily and gradually increase their proficiency with the program without being overwhelmed by options. On the other hand, advanced users can set the highest user level and use the program in its entirety from the start.

1.1.2 Darklang

Darklang [4] is a cloud-based low-code programming system for building web application backends created by Paul Biggar. The original system's development was discontinued in 2023 and renamed Darklang-classic.

The following summary follows the official Darklang documentation[1]. The Darklang project consists of a low-code editor and a programming language. It also provides tools for creating persistent storage and for deployment. The editor provides the service's main functionality. It consists of a canvas that displays the created application elements as draggable boxes as seen in Figure 1.2. Each box provides options to modify the element. These options include different menus and input fields. Some elements can be modified using the Darklang programming language to implement custom behavior. The user can create program elements of the following categories:

- HTTP handlers - definition of API endpoints
- Persistent storage - database creation and modification
- Workers - processing of background tasks

[1] `https://docs.darklang.com/` [visited on 2024-04-22][online]

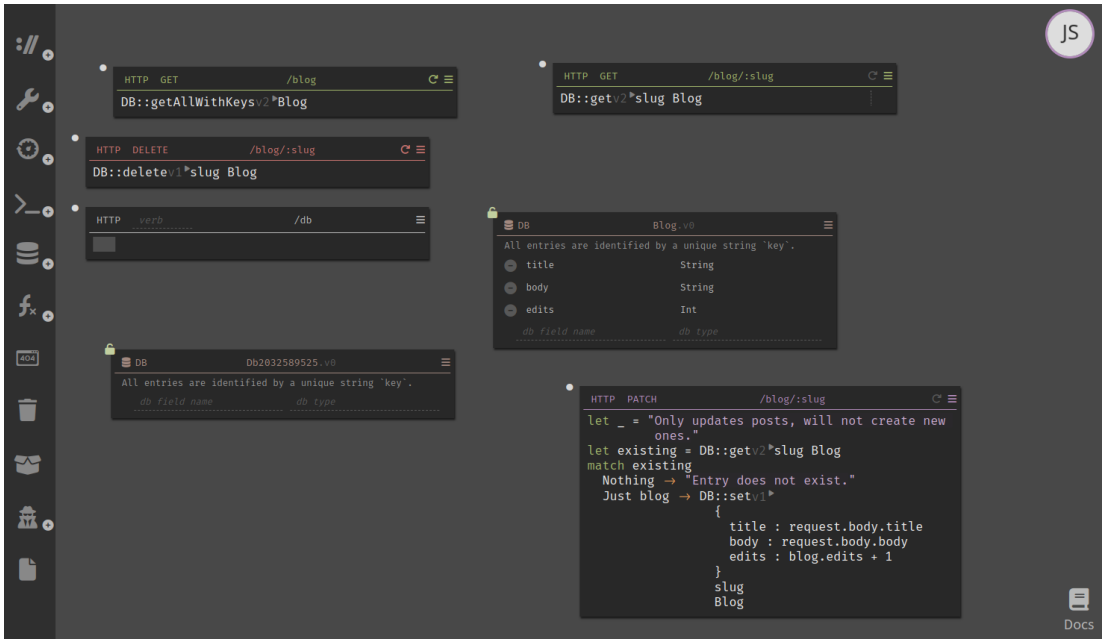

Figure 1.2 Example of Darklang's drag-and-drop user interface.

- Cron jobs - scheduled jobs with custom behavior
- REPL - a general-purpose programmable element
- Functions - a reusable element with custom behavior

By combining these elements, the users can create application backends of varying complexity. The system automatically redeploys the entire application after the user creates and modifies a particular element. This allows for an interactive style of development where changes made to the system are almost immediately visible.

1.1.3 Output-directed programming

Output-directed programming is described by Hempel et al. [5] as a programming paradigm where users construct plain text programs using mouse-based operations on the program's output. The difference between this paradigm and other low-code programming approaches is that the source code is still the main representation of the program. When making a change using the mouse-based operations is too hard or impossible, the user can still change the output by directly changing the source code. This approach is made possible thanks to the use of *live synchronization* and *trace-based program synthesis* as defined by Chugh et al. [6]

One example of a programming system that demonstrates this paradigm is called *Sketch-and-Sketch* [5, 6]. It is a browser-based application that provides a bimodal programming environment. It allows creating and modifying Scalable Vector Graphics (SVG) by directly manipulating the program's SVG output. The application consists mainly of an editor window for the source code, a canvas window displaying the program's output, and a button to run the program. After clicking on the canvas, the output is overlaid by context menus and *widgets*, allowing users to modify the program's *sub-expressions* and *intermediate values* as seen in Figure 1.3.

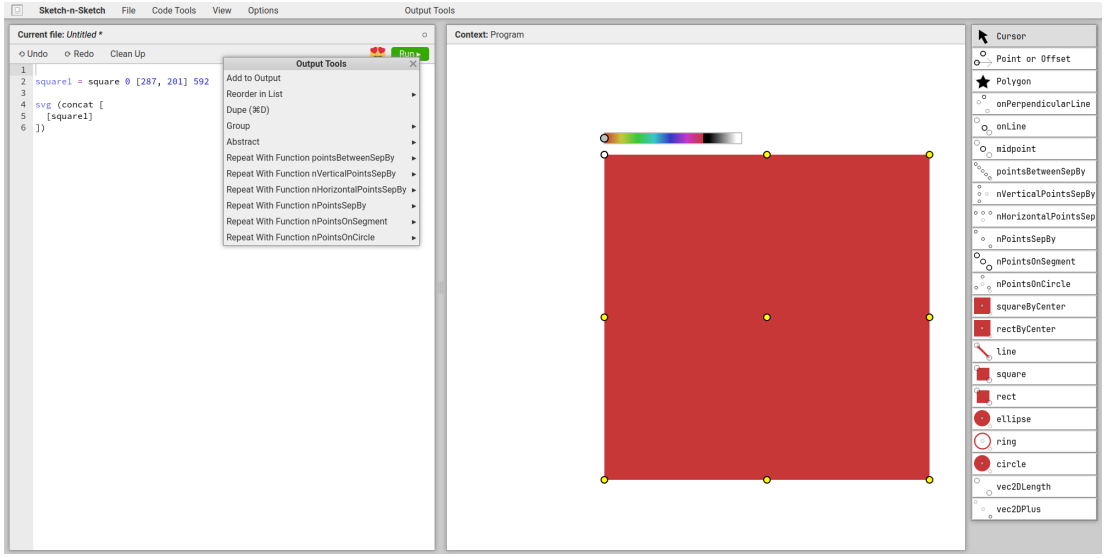

Figure 1.3 Example of Sketch-and-Sketch's user interface.

Sub-expressions refer to a certain syntactic scope of the program and the user can select and modify it. *Intermediate values* are values produced at the intermediate execution steps of the program. The program provides widgets for different types of intermediate values such as *offset*, *point*, or *list* widgets. Widgets can take the form of draggable boxes or input fields.

Another programming system implementing the direct manipulation paradigm is *Transformic* introduced by Schreiber et al. [7]. It is a functional interpretation of the Morphic GUI Framework providing direct-manipulation functionality.

1.2 Web development

This section serves as a description of web development technologies and tools. We will present the Elm programming language, and describe tools which enable the development of browser-based applications in F#, such as the F# to JavaScript compiler Fable and the Elmish architecture.

1.2.1 HTML

//TODO:

1.2.2 JSON

//TODO:

1.2.3 Elm

Elm [8] is a functional programming language presented by Czaplicki and Chong [9]. It allows the creation of responsive graphical user interfaces by employing the *functional reactive programming* approach. The functional reactive programming approach applies pure functional programming paradigms to time-varying values. Time-varying values can represent the input and output of the GUI or other information

channels, such as server requests. The applications to time-varying values are known as *signals*. Elm improves upon other functional reactive programming languages by assuming that the signals are discrete. This means that it avoids unnecessary recomputation when the signals are unchanged. Discrete change of a signal is called an event. Events trigger the recomputation and the application is updated. Elm also provides tools and abstractions to enable asynchronous computations.

The main categories of GUI elements in Elm are *forms* and *elements*. *Elements* represent a *rectangular* GUI element with properties such as width and height. Elements can contain data in a form of text, video or an image and can be composed together. Forms on the other hand represent two dimensional elements of arbitrary shapes with modifiable properties such as color and texture.

Reactive GUIs can be implemented by combining GUI elements and input signals. Elm provides a range of primitive signals such as "Mouse.clicks" which triggers on mouse click. Some primitive signals may require arguments for their constructor such as the "Window.dimensions" signal.

1.2.4 Fable

Fable [10] is a compiler capable of compiling programs written in F# to multiple target languages. One of the available target languages is JavaScript and the compiler produces human-readable and formatted JavaScript code. This enables the creation of web applications in a type-safe functional programming language while also enabling the use of tools and libraries from the wider JavaScript ecosystem.

The compiler supports the use of some types from .NET Base Class and FSharp.Core libraries [11] and tries to compile the types to native JavaScript whenever possible.

1.2.5 Elmish

Elmish or *The Elm Architecture*[12] is an architecture for building applications inspired by the Elm programming language. The architecture follows the *Model-View-Update* pattern or *MVU*. The main emphasis of the architecture is to extensibily, type-safely, and declaratively create reactive applications with graphical user interface.

The *Model* is a data-structure which keeps the state of the entire application. It holds data and other necessary values that the application uses. The data-structure is usually immutable. This means that making changes to the state results in a creation of an updated copy of the state which becomes the new state.

View is a function which takes the current state of the application and creates a graphical user interface. The interface usually consists of multiple GUI elements displaying the application state.

Update is a function which updates the model based on the inputs from the graphical user interface which calls the update function with a certain *Message*. Messages are similar to events in the Elm language. Each message represents a certain change to the model. The update function updates the model based on the provided message and the current application state. The message type is usually represented as a *disciminated-union*. This enables the update function to use pattern-matching expressions to efficiently update the model.

2 Walktrough

2.1 To-do application

3 Design principles

This chapter describes the main design principles behind the data-driven low-code programming system for creating web applications shown in Chapter 2. The main design principles discussed include:

1. Data-driven approach
2. Low-code editing
3. Code generation capability

3.1 Data-driven approach

The main general design principle of the proposed programming system is the *data-driven* approach in context of creating web application UI elements. We define the general approach as follows:

Definition 2 (Data-driven approach)**.** *A software development approach where the creation of a particular software artifact is dependant on* ***concrete data*** *provided to the system.*

Meaning that the development process is tied to the specific data provided. In the context of creating web application UI elements, some data can be used only for certain UI elements as to maintain the correct semantic meaning and functionality of the elements. Another definition which could be used to describe our approach is the following:

Definition 3 (Data to UI)**.** *Defined by Sahay et al. [13] as "A data-driven approach that starts from data modeling and then builds the user interface of the application followed by the specification of business logic rules and workflows."*

This definition also includes the concepts of data modeling and specification of business logic rules and workflows but we only focus on the aspect of user interface creation in this thesis. The main difference between our programming system and other systems described by Sahay et al. [13] as *Data to UI* such as *Mendix* [14] is that creating the user interface is directly tied to the data provided to the application. Usually, the *Data to UI* systems provide options to create pre-made UI elements, which may then reference and use the provided data. Whereas our proposed system creates the UI elements based on the data itself which then reference the provided data.

The system analyzes the provided data and provides development options based on its structure and properties. The analysis can be tailored to the specific aims of the programming system. For example, the proposed application for creating web applications can analyze the data and map it to specific HTML elements based on its type and structure. Usually, the developer performs this data analysis and creates the GUI elements by hand. Using this approach, the developer can offload some of this work onto the system and focus on other aspects of the development process.

The data analysis can employ various techniques, such as data structure and data type recognition. One option is that the system performs the analysis once after the

developer provides data to the system, and the system provides options based on this single analysis. Alternatively, the system can perform it continuously throughout the development process and provide development options based on the already created application elements, similar to Output-directed programming described in Section 1.1.3. The system could also use various machine-learning techniques to improve the data analysis.

Another valuable property of systems employing this approach is their ability to be used in conjunction with other existing technologies, mainly those focused on data modeling and retrieval. For example, the developer can use the proposed system employing this approach together with existing technologies such as *GraphQL* [15]. Using GraphQL, the developer can specify the data they wish to use and its structure. This data can then be provided to the system and used to create the desired web application.

3.2 Low-code editor

The second design principle is the concept of a *low-code editor*. Low-code editors usually provide a graphical user interface through which the developer interacts with the system and develops the desired application. The system provides contextual menus to aid in the creation of the application. Various existing low-code programming editors were presented previously in section 1.1.

The visual design and functionality of the system is inspired by the "what you see is what you get" (abbreviated as *WYSIWYG*) editors presented by Yang et al. [16]. This sub-category of low-code editors displays the application's output as if running inside its native environment. This design allows the system to update the output every time a change is made to the program and display its result, allowing for a more dynamic development loop [6].

Our prototype system interweaves the contextual menus with the application's output, as seen in chapter 2. Each menu is placed next to the corresponding element, which it can modify. The developer can also hide these menus and show only the application's output. The design of the contextual menus was influenced by menus in currently existing programming systems, such as property menus from the *Visual studio* editor.

3.3 Code generation

The last design principle is the ability to generate a plain–text representation of the created application. This ability addresses some of the concerns and main pitfalls of many low-code programming systems, as stated by Pinho et al. [1]. These concerns include *Interoperability issues* and *Vendor lock-in*. Both are mitigated as the user can download the already created part of the application in its textual representation and continue its modification in another programming system.

Our prototype implementation also provides a code generation capability. The type system of the application represents the created parts of the application in a form that allows translation of the created application to various programming languages and other technologies. The prototype system can target the *PHP* programming language but could be extended to provide a wider selection of target technologies.

4 Core logic

This chapter describes the core logic of the programming system. Firstly, we describe the hole-based approach applied to the creation of elements. After that, we describe the type system, mainly the internal representation of the UI elements and necessary operations on these types. Then, we describe mapping the input data types to specific types representing the UI elements. We present two ways of combining the created UI elements with the corresponding data. Lastly, we describe one possible way of displaying a preview of the created UI elements and the modification menus.

4.1 Hole-based approach

The primary motivation of this approach is to allow *incremental creation* of UI elements. The elements are created based on the provided data and its structure. Each element corresponds to a data value. We need the ability to represent data values that have yet to be used to create UI elements. To address this need, we define a type named *Hole* as follows:

Definition 4 (Hole type definition)**.** *A Hole is a placeholder type representing a UI element that is yet to be created despite its existing corresponding data. It serves as a temporary stand-in during the incremental construction of a user interface.*

We then define the incremental creation process as a *sequence of discrete opreations*. Each operation is either a *modification* of an existing UI element or a *replacement* of a Hole element with a new UI element based on the corresponding data. This discrete approach allows the system to perform different tasks after each operation. These tasks include updating the UI element preview, analyzing the input data, or providing new modification menus and options. The system could also analyze the existing UI elements and other operations.

4.2 Type system and Operations

4.2.1 JSON type

The creation process starts by providing *JSON* data to the system. In order to use this data, we parse it and create an internal representation. We define a discriminated union type named *JSON*. This type is the foundation for the Abstract Syntax Tree (AST) representation of the JSON data. The type is defined as a as follows:

Program 1 JSON type

```
type Json =
  | JNumber of float
  | JString of string
  | JBool of bool
  | JNull
  | JArray of Json list
  | JObject of Map<string, Json>
```

Each node represents a JSON value in the input data. The types of nodes can be divided into two categories:

- **Collections:** The first category contains types representing a *collection* of other values. This category includes the types *JObject* and *JArray*. We define the two collection types as follows:
  - **JObject:** It is based on the JSON Object type and represents a collection of different JSON types. The original ordering of the inner elements is *ignored*.
  - **JArray:** It is based on the JSON List type and represents a collection of JSON values of the *same type and structure*. The original ordering of the inner elements is *preserved*.
- **Primitives:** The second category contains types representing data primitives such as numerical values, boolean values, a string of text, or the null value.

4.2.2 RenderingCode type

To facilitate the creation of UI elements based on the JSON type, we define a type named *RenderingCode*. The *RenderingCode* is a discriminated union type used to represent the UI elements. Similarly to the JSON type, each case represents a type of an HTML element with a corresponding mapping to the JSON type. This type is the foundation for the *Abstract Syntax Tree* representation of UI elements.

Program 2 RenderingCode type

```
type RenderingCode =
  | HtmlElement of
      tag: Tag *
      attrs: Attributes *
      innerValue: InnerValue
  | HtmlList of
      listType: ListType *
      elements: RenderingCode list
  | HtmlObject of
    objectType : ObjectType *
    elements: RenderingCode list
  | Hole
```

The case types are defined as follows:

- **HtmlElement:** Represents a single HTML element. It consists of:
  - *Tag*: Denotes the HTML tag (e.g., div, p, span).
  - *InnerValue*: Represents the content of the element, defined as:

```
type InnerValue =
  | Data
  | Empty
  | Constant of string
```

Where *Data* indicates dynamic content from JSON, *Empty* represents an empty element, and *Constant* holds static, developer-defined content.

- **HtmlList:** Represents a collection of similar HTML elements corresponding to JSON arrays. It includes:
  - *ListType*: Specifies the list type (e.g., unordered, ordered, table).
  - *itemCodes*: A list of RenderingCode elements representing list items.
  - *Attributes*: A collection of key-value pairs representing HTML attributes. The Attribute type is defined as follows:

    ```
    type Attribute = string * InnerValue
    ```

    It consists of a key of a type string and an value of a type InnerValue. This allows selecting the value of an Attribute as a value from the input data.

- **HtmlObject:** Represents a collection of diverse HTML elements derived from JSON object. It consists of:
  - *ObjectType*: Defines the type of the object (e.g., div, span, section).
  - *Elements*: A list of RenderingCode elements representing the object's contents.

- **Hole:** Represents a placeholder in the UI element structure as defined in Section 4.1, allowing for dynamic element creation. The Hole is a discriminated union type

- //TODO: CustomWrapper and Javascript

4.2.3 Data Mapping: JSON to RenderingCode

The process of transforming JSON data into UI elements requires systematic mapping between JSON and RenderingCode types. This mapping forms the core of our UI generation system, allowing us to convert arbitrary JSON input into a structured representation of UI elements. Each JSON type has a corresponding representation in RenderingCode:

- JObject maps to HtmlObject

- JArray maps to HtmlList

- JNull, JString, JNumber, and JBool map to HtmlElement

Th

We define a mapping function named *recognizeJson* as follows:

Program 3 RenderingCode type

```
let recognizeJson (json: Json) =
    match json with
    | JArray array ->
        //create a hole for each element of the array
        HtmlList containing the holes as elements
    | JObject obj ->
        //create hole for each element of the object
        HtmlObject containing the holes as elements
    | JNull -> Hole(UnNamed)
    | _ -> HtmlElement
```

4.3 Creation process

4.4 Referencing input data

4.4.1 Structual referencing

4.4.2 Selector based approach

5 Implementation

5.1 Technologies

5.2 Design

5.3 User interface

5.4 Summary

6 Discussion

6.1 7GUIs benchmarks

6.2 Structual referencing vs Selectors

6.3 Potential drawbacks of the approach

6.4 Future work

6.5 Summary

Conclusion

Bibliography

1. Pinho, Daniel; Aguiar, Ademar; Amaral, Vasco. What about the usability in low-code platforms? A systematic literature review. *Journal of Computer Languages*. 2023, vol. 74, p. 101185. issn 25901184. Available from doi: 10.1016/j.cola.2022.101185.
2. Leonard, Peter. *Macworld Expo 1987 Boston*. 1987. Available also from: http://32by32.com/macworld-expo-1987-boston/.
3. Goodman, Danny. *The complete HyperCard handbook*. 2nd ed. Toronto ; New York: Bantam Books, 1988. The Macintosh performance library. isbn 9780553345773.
4. *Darklang* [online]. [visited on 2024-04-22]. Available from: https://darklang.com/.
5. Hempel, Brian; Lubin, Justin; Chugh, Ravi. Sketch-n-Sketch: Output-Directed Programming for SVG. In: *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology*. New Orleans LA USA: ACM, 2019, pp. 281–292. isbn 9781450368162. Available from doi: 10.1145/3332165.3347925.
6. Chugh, Ravi; Hempel, Brian; Spradlin, Mitchell; Albers, Jacob. Programmatic and direct manipulation, together at last. *ACM SIGPLAN Notices*. 2016, vol. 51, no. 6, pp. 341–354. issn 0362-1340. Available from doi: 10.1145/2980983.2908103.
7. Schreiber, Robin; Krahn, Robert; Ingalls, Daniel H. H.; Hirschfeld, Robert. *Transmorphic: Mapping direct manipulation to source code transformations*. Potsdam: Universitätsverlag, 2017. Technische Berichte des Hasso-Plattner-Instituts für Softwaresystemtechnik an der Universität Potsdam. isbn 9783869563879.
8. *Elm* [online]. [visited on 2024-04-29]. Available from: https://elm-lang.org/.
9. Czaplicki, Evan; Chong, Stephen. Asynchronous functional reactive programming for GUIs. *ACM SIGPLAN Notices*. 2013, vol. 48, no. 6, pp. 411–422. issn 0362-1340, issn 1558-1160. Available from doi: 10.1145/2499370.2462161.
10. *Fable* [online]. [visited on 2024-04-29]. Available from: https://fable.io.
11. *Fable compatibility* [online]. [visited on 2024-04-29]. Available from: https://fable.io/docs/javascript/compatibility.html.
12. *Elm architecture* [online]. [visited on 2024-04-29]. Available from: https://guide.elm-lang.org/architecture/.
13. Sahay, Apurvanand; Indamutsa, Arsene; Di Ruscio, Davide; Pierantonio, Alfonso. Supporting the understanding and comparison of low-code development platforms. In: *2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. Portoroz, Slovenia: IEEE, 2020, pp. 171–178. isbn 9781728195322. Available from doi: 10.1109/SEAA51224.2020.00036.
14. *Mendix* [online]. [visited on 2024-05-16]. Available from: https://www.mendix.com/.
15. *GraphQL* [online]. [visited on 2024-05-16]. Available from: https://graphql.org/.

16. Yang, Fan; Gupta, Nitin; Botev, Chavdar; Churchill, Elizabeth F; Levchenko, George; Shanmugasundaram, Jayavel. WYSIWYG development of data driven web applications. *Proceedings of the VLDB Endowment*. 2008, vol. 1, no. 1, pp. 163–175. ISSN 2150-8097. Available from DOI: 10.14778/1453856.1453879.

List of Figures

1.1 Selection of a user level in the Hypercard preferences menu. 10
1.2 Example of Darklang's drag-and-drop user interface. 11
1.3 Example of Sketch-and-Sketch's user interface. 12

A Attachments

A.1 First Attachment